THORRIJA

Engineering Showcase — Qué hice y cómo funciona

Cinco modelos computacionales deterministas reconstruidos desde código fuente. Todo se calcula en el momento. Nada está hardcodeado. Misma entrada, misma salida.

Autor: Thorrija (Rubén N.)
España
Noviembre 2026
GitHub: github.com/ThorrijaDEV

1. Qué construí

Construí un Engineering Showcase: una aplicación web con cinco laboratorios interactivos. Cada laboratorio es un modelo computacional determinista reconstruido a partir del código fuente de proyectos originales. No ejecuto los proyectos originales dentro de las demostraciones. Todo lo que se ve en pantalla se calcula en el momento a partir de tu entrada.

Los cinco laboratorios están en `/home/thorrija/showcase/` y los modelos en `/home/thorrija/models/`.

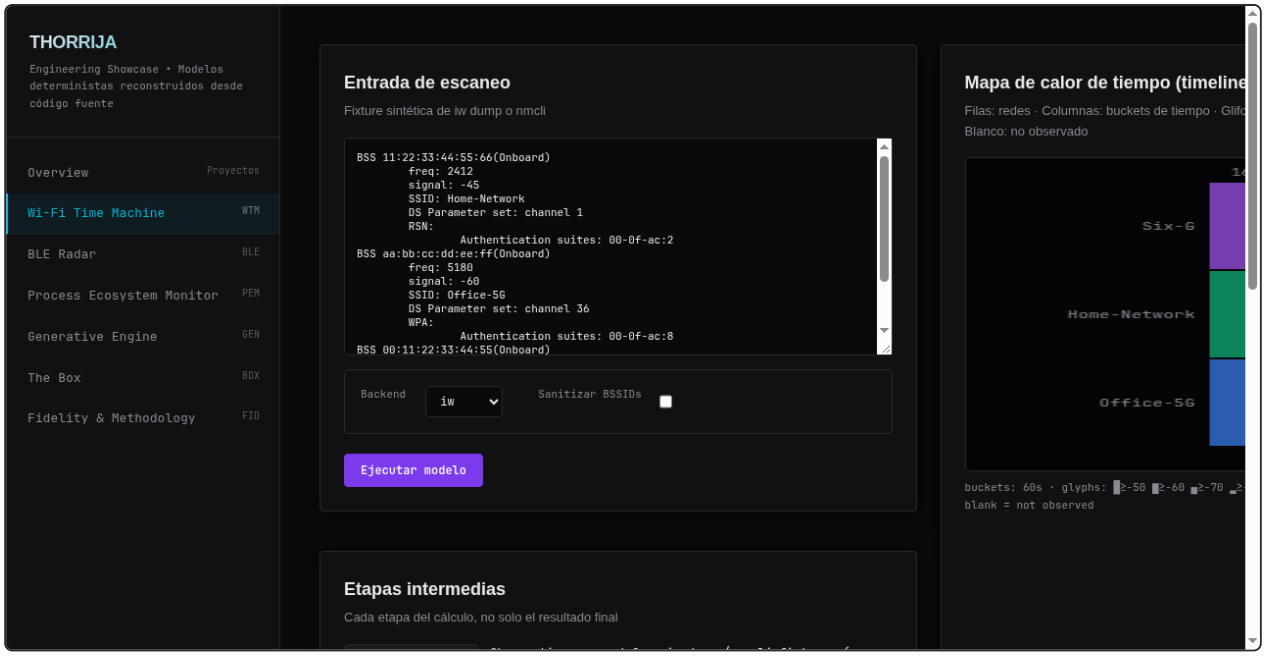
LABORATORIO	QUÉ HACE	CÓDIGO FUENTE ORIGINAL
Wi-Fi Time Machine	Historial de entornos Wi-Fi: escanea, mapea canales, muestra calor del tiempo	wifi-time-machine
BLE Radar	Radar de dispositivos Bluetooth: ciclo de vida, fuerza de señal, ángulo por hash de MAC	ble-radar
Process Ecosystem Monitor	Vida de procesos Linux: nacimiento, muerte, CPU por jiffies, árbol de procesos	process-ecosystem-monitor
Terminal Generative Engine	Arte generativo determinista: flowfield, plasma, attractor	terminal-generative-engine
The Box	Sustrato relacional: seis reglas de reescritura, linaje y fusiones	the-box

2. Wi-Fi Time Machine

Qué hace: Toma un volcado de escaneo Wi-Fi (formato iw o nmcli) y lo convierte en una línea de tiempo de calor: qué redes aparecieron en qué momentos, con qué fuerza y en qué canales.

Cómo funciona

- Parsea bloques BSS del salida de `iw scan dump` o salida de nmcli.
- Deduplica por BSSID: si la misma red apareció dos veces, se queda con la señal más fuerte.
- Mapea frecuencia a canal 802.11 (2.4 GHz, 5 GHz, 6 GHz).
- Infere seguridad (WPA2, WPA3) de los suites de autenticación.
- Construye un mapa de calor tiempo × buckets: cada celda muestra la señal media de esa red en ese intervalo.
- Exporta JSON/CSV con direcciones MAC sanitizadas (SHA-256 truncado).



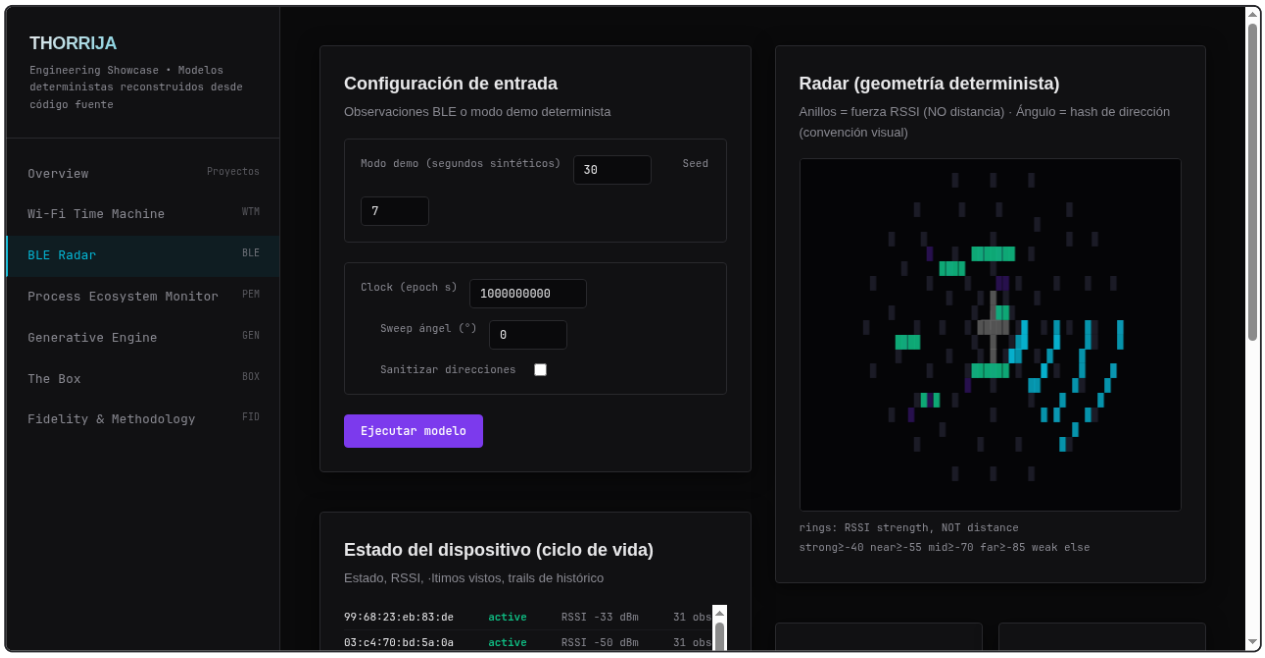
Wi-Fi Time Machine: entrada de escaneo, mapa de calor de tiempo, actividad por canal, sparkline de señal.

3. BLE Radar

Qué hace: Detecta dispositivos Bluetooth LE cercanos, les sigue el ciclo de vida (aparecido, activo, desvaneciendo, desaparecido) y los dibuja en un radar donde el ángulo es un hash de su dirección MAC y el anillo es la fuerza de señal RSSI (no distancia).

Cómo funciona

- Recibe observaciones BLE: dirección MAC, nombre, RSSI, timestamp.
- Suaviza el RSSI con EMA (alpha = 0.4): cada lectura nueva mezcla 40 % nueva, 60 % anterior.
- Clasifica la fuerza en bandas: strong (≥-40), near (≥-55), mid (≥-70), far (≥-85), weak.
- Calcula un ángulo estable a partir del hash SHA-256 de la dirección MAC. Es una convención visual, no una orientación real: BLE no da orientación.
- Máquina de estado: un nodo desaparece si no recibe nada en 30 segundos.



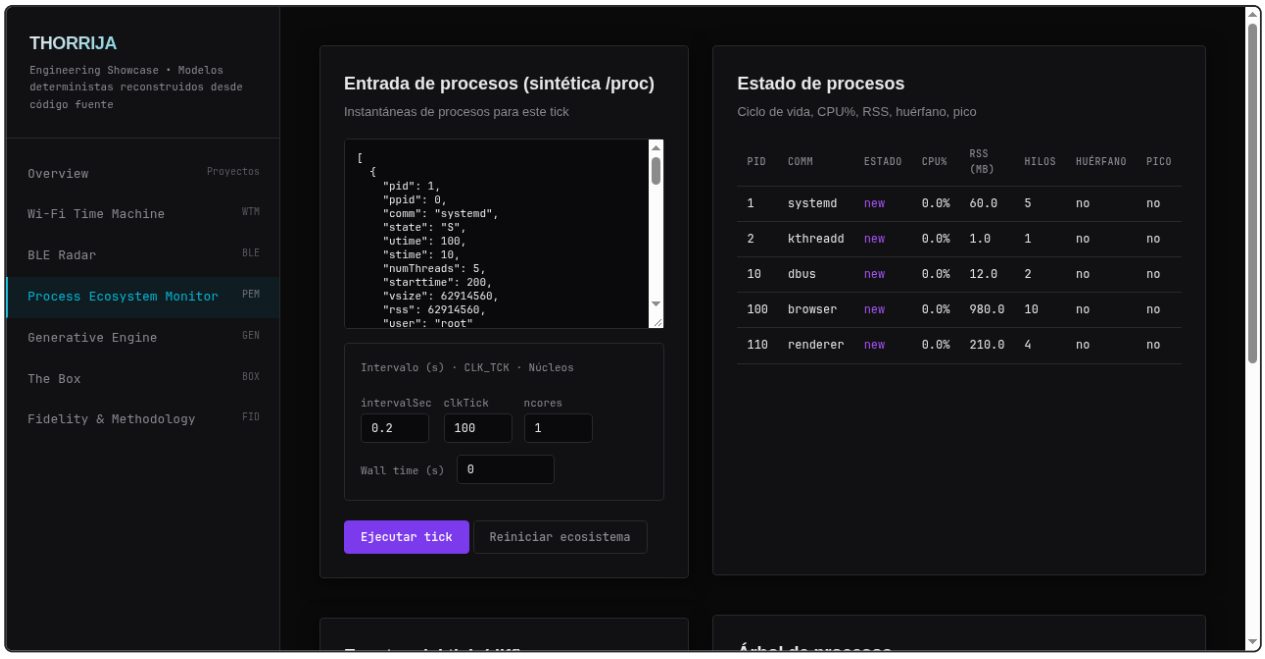
BLE Radar: dispositivos activos, anillos de fuerza RSSI, línea de barrido, historial de dispositivos.

4. Process Ecosystem Monitor

Qué hace: Toma instantáneas de procesos estilo /proc/[pid]/stat y construye un árbol de procesos: quién es padre de quién, quién consumió más CPU, quiénes murieron, quiénes reutilizaron PID.

Cómo funciona

- Parsea /proc/[pid]/stat (incluyendo nombres con paréntesis anidados), /proc/[pid]/statm, /proc/stat, /proc/meminfo, /proc/loadavg.
- Calcula CPU% del sistema: busy delta / total delta de /proc/stat.
- Calcula CPU% de cada proceso: delta de (utime+stime) jiffies dividido por (intervalo × CLK_TCK × núcleos).
- Identidad del proceso: (pid, starttime). Si el starttime cambia, es un proceso nuevo que reutilizó el PID del anterior.
- Árbol de procesos: se construye desde ppid. Los huérfanos se reasignan a init (pid 1).



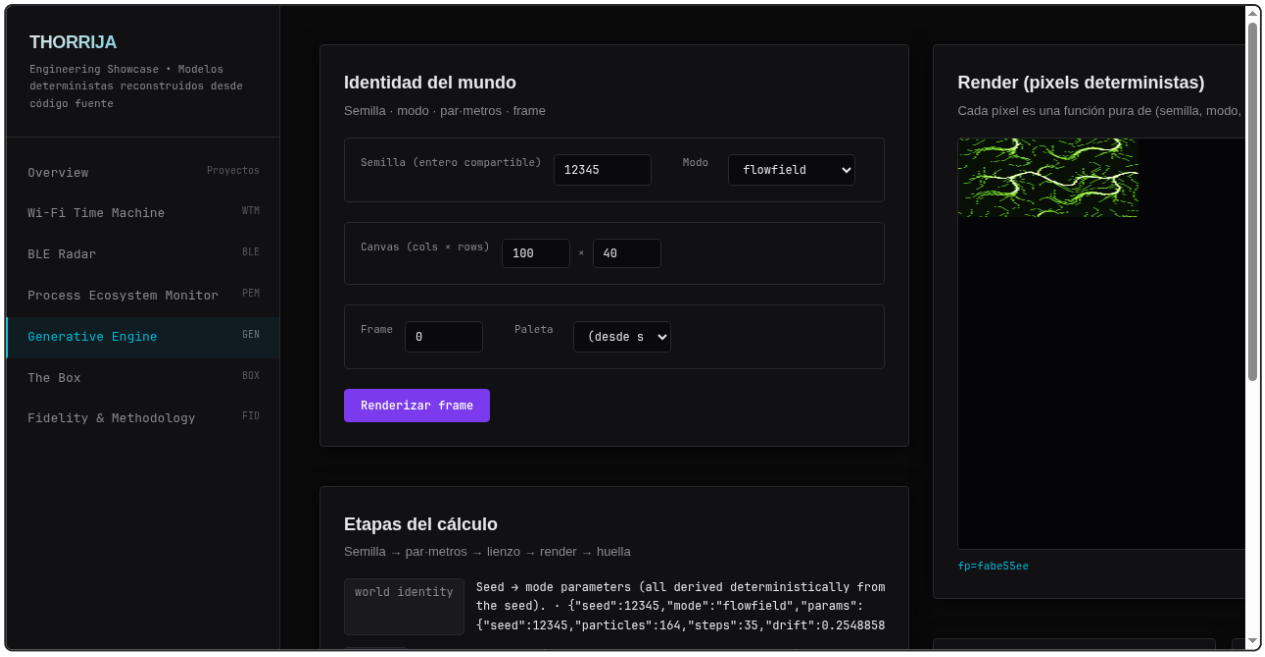
Process Ecosystem Monitor: instantáneas de procesos, eventos del tick, árbol de procesos, tabla de estados.

5. Terminal Generative Engine

Qué hace: Genera arte generativo determinista a partir de una semilla: flowfield, plasma o attractor. Misma semilla + mismo frame = mismo resultado siempre. Puede saltar a cualquier frame sin calcular los anteriores (O(1)).

Cómo funciona

- **RNG sin estado:** splitmix64-style. Cada valor es una función pura de (semilla, frame, salt, índice). No hay estado oculto.
- **Paletas:** colores continuos interpolados linealmente entre paradas.
- **xterm-256:** mapeo de color.
- **Framebuffer:** W×H pixel canvas, codificación de píxeles de medio bloque terminal.
- **Modos:** flowfield (campo de ángulo analítico de senos), plasma (fórmulas cerradas de color), attractor (mapa extraño de Clifford/Dejong).
- **Huella del mundo:** SHA-256 sobre modo + frame + parámetros + píxeles. Mismo conjunto de parámetros, misma huella.



Terminal Generative Engine: semilla, modo, canvas de píxeles deterministas, huella del mundo.

6. The Box

Qué hace: The Box es un sustrato relacional: un grafo de arcos y nodos donde seis reglas de reescritura modifican la estructura. A partir de esas reglas emergen linaje (quién engendró a quién) e identidad (quiénes se fusionaron). Nada en el kernel almacena identidad, linaje o generación: son propiedades derivadas de las etiquetas de los arcos.

El sustrato (kernel)

- **Nodo:** {id, tag, payload (vector de números), born}
- **Arco (relación):** {id, src, dst, tag, peso, clock}
- **Ids compartidos:** nodos y arcos comparten el mismo numerador de ids. Cualquier id puede referirse a un nodo o a un arco.
- **Segundo orden:** un arco puede tener otro arco como extremo.

Las seis reglas

REGLA	QUÉ HACE
GATHER	Dos partidarios del mismo hub se conectan directamente.
BOND	El ancla y un vecino de salida que comparten dos vecinos de entrada se enlazan.
TAG_PROP	El ancla copia la etiqueta de un vecino de salida.
QUIET	Un nodo que no ha recibido nada en 30 pasos se marca como QUIET (tag 5).
DESCEND	Un nodo QUIET con espacio genera un hijo con su etiqueta y payload con pequeño jitter.
FUSE	Dos hermanos (mismo padre vía DESCEND) con misma etiqueta y payload cercano se identifican: el gemelo se retira y un arco EQUIV queda como testigo de la fusión.

Motor

- Cuatro políticas de selección: canonical (sin semilla, sitio estructuralmente más pequeño), uniform, weighted, roundrobin.
- Cada paso: enumera todos los sitios posibles, selecciona uno, aplica la regla, registra el evento.

Identidad y linaje (derivados, no campos del kernel)

- **Descenso:** quién engendró a quién, a partir de arcos DESCEND.
- **Profundidad de linaje:** cuántos niveles tiene cada nodo en el árbol de descendencia.
- **Identidad:** qué nodos se fusionaron, a partir de arcos EQUIV (testigos de fusión).
- **Fused fraction:** qué porcentaje de nodos participó en alguna fusión.

Métricas estructurales

- **degree_tables:** grados de entrada y salida sobre arcos nodo-nodo; coeficiente de Gini.
- **components:** componentes conexos (union-find).
- **beta0_beta1:** Euler-exacto sobre el grafo de incidencia; beta1 = ‖arcs‖ − ‖objects‖ + beta0.

- **component_size_profile:** n, max, mean, entropy, largest_frac.
- **modularity:** determinista Louvain (un solo nivel).

Observables nombrados (14)

- n_nodes, n_arcs, arc_density, beta0, beta1
- mean_in_degree, mean_out_degree, max_in_degree, gini_in_degree
- largest_component_frac, tag_entropy
- mean_lineage_depth, max_lineage_depth
- n_identity_classes, n_fusion_classes, fused_nodes, fused_fraction

Null models

- **degree-preserving rewire:** doble intercambio de bordes; preserva cada grado de entrada y salida; destruye estructura de orden superior (Milo et al. 2002).
- **tag permutation:** preserva el multiset de etiquetas; destruye qué nodo lleva qué etiqueta.
- **homogeneous:** etiquetas borradas, pesos normalizados; conserva topología.

p-Values

P-valor empírico de un lado con corrección +1:

```
p = (count(null ≥ real) + 1) / (null_count + 1)
```

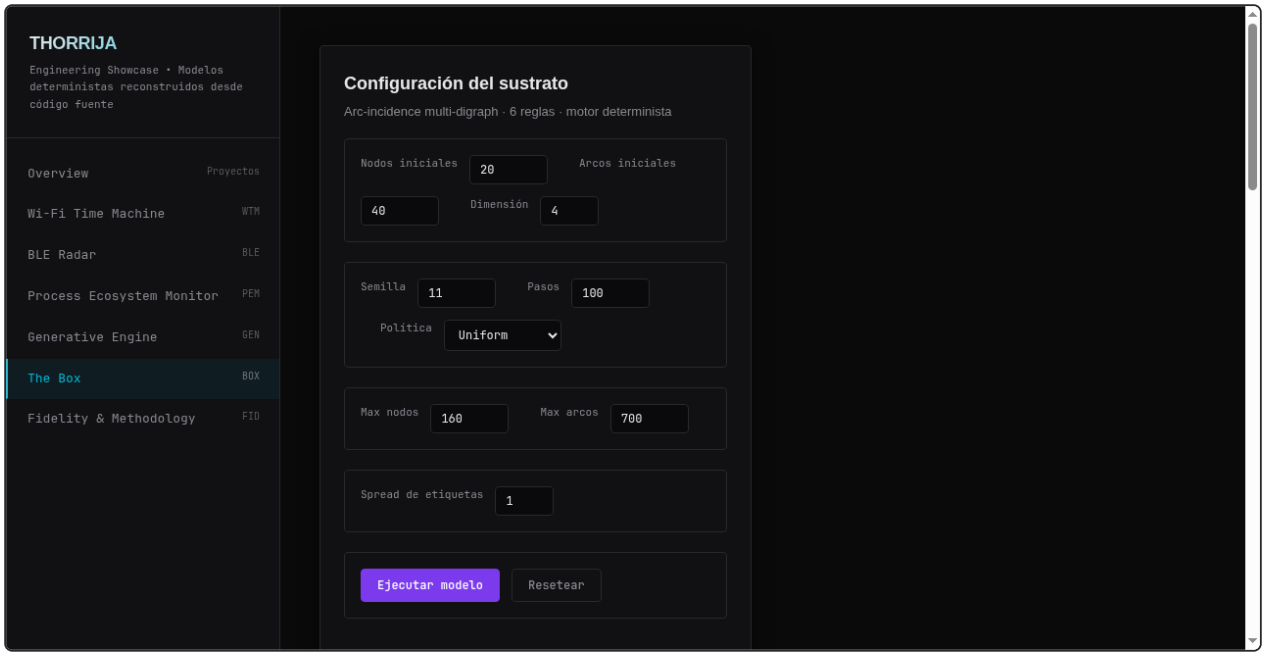
Resolución del piso: 1/(null_count + 1). Si el piso supera alpha (0.01), el test no es attainable.

Provenance

El fingerprint del experimento es BLAKE2b-16 sobre: name, seed sum, steps, substrate_seed, ruleset_id, ruleset_hash, config, fingerprint_version. Dos records con el mismo fingerprint son el mismo experimento.

Limitaciones

- **The Box: fidelidad Nivel 3.** El BLAKE2b está reimplementado en TypeScript puro; no es byte-compatible con Python hashlib.blake2b. Los hashes de grafo, unit_hash y huellas de experimento difieren de Python. Las propiedades estructurales (nodos, grados, linaje, observables) son idénticas.
- Wi-Fi Time Machine y BLE Radar no acceden a hardware real; usan datos sintéticos.
- Terminal Generative Engine: no tiene exportación de archivo (solo arrays de píxeles).
- No hay base de datos SQLite; todo en memoria.



The Box: sustrato arc-incidente, nodos por etiqueta, arcos por peso, nodos retirados desvanecidos.

7. Cómo ejecutarlo

```
# Showcase (web)
cd /home/thorrija/showcase
npm run dev      # servidor de desarrollo en http://localhost:3000
npm run build    # construcción para producción en dist/
npm run preview  # previsualizar el build

# Modelos (TypeScript / Jest)
cd /home/thorrija/models
npm test         # 102 pruebas, todas pasan
```

No necesitas claves de API, hardware Wi-Fi o Bluetooth, ni /proc del sistema anfitrión. Todo funciona con datos sintéticos.

8. Limitaciones (honesto)

- **The Box: fidelidad Nivel 3.** El BLAKE2b está reimplementado en TypeScript puro; no es byte-compatible con Python hashlib.blake2b. Los hashes de grafo, unit_hash y huellas de experimento difieren de Python. Las propiedades estructurales (nodos, grados, linaje, observables) son idénticas.
- Wi-Fi Time Machine y BLE Radar no acceden a hardware real; usan datos sintéticos.
- Terminal Generative Engine: no tiene exportación de archivo (solo arrays de píxeles).
- No hay base de datos SQLite; todo en memoria.

Repositorios

- github.com/ThorrijaDEV/showcase — Showcase web
- github.com/ThorrijaDEV/models — Modelos deterministas
- github.com/ThorrijaDEV/Portafolio — Portafolio